

Grafos de Conhecimento Centrados em Sessão HTTP para Detecção Explicável de DDoS

Natanael Oliveira^{a,*}, Arthur Kobielski^a, Marcos Luna^a

^a *Universidade Federal do Rio Grande do Sul, Porto Alegre, Brasil*

Abstract

Campanhas coordenadas de DDoS de Camada 7 sobre HTTP, em particular a família *Slow HTTP DDoS distribuído*, sustentam taxas baixas por sessão e passam por defesas baseadas em assinatura, limiares estatísticos e quotas por IP, sessão ou *token*. O sinal de ataque não está em nenhuma sessão isolada, mas na correlação estrutural entre sessões que compartilham alvo, *fingerprint* TLS, prefixo de IP ou identidade reaproveitada. A literatura de detecção reduz a sessão a um vetor numérico antes do classificador e descarta exatamente essa estrutura, de modo que o detector não articula *por que* uma sessão é suspeita e perde o padrão *cross-session* que define a campanha. Trabalhos recentes em grafos de conhecimento aplicados à segurança de rede combinam o grafo com classificadores neurais e explicação pós-hoc, mostrando ganhos em explicabilidade, mas tratam o tráfego em nível de nó de rede (porta, IP) e não modelam a sessão HTTP como entidade semântica raciocinável; tampouco acoplam o veredicto a uma política de mitigação derivada do próprio grafo. Este artigo propõe um arcabouço que modela a sessão HTTP como objeto semântico de primeira classe em OWL, com relações tipadas para identidade do cliente, *endpoint* alvo, sinais comportamentais e mitigação aplicável, e alinhamento a STIX 2.1 e MITRE ATT&CK. Requisições são elevadas em tempo de execução a um grafo no qual regras simbólicas em SPARQL/SWRL produzem veredictos com cadeias de evidência percorrendo múltiplas sessões. O discriminador do *cluster* detectado é usado para derivar automaticamente o escopo da mitigação aplicada, de modo a preser-

*Autor correspondente

Email addresses: natanael.oliveira@inf.ufrgs.br (Natanael Oliveira), arthur.kobielski@inf.ufrgs.br (Arthur Kobielski), marcos.luna@inf.ufrgs.br (Marcos Luna)

var o tráfego legítimo. A avaliação concentra-se na família Slow HTTP DDoS distribuído como caso experimental, parametrizada pelo *grau de distribuição* da campanha. A ablação isola o ganho do raciocínio em nível de sessão em relação a detectores que consomem as mesmas *features* agregadas, e introduz uma métrica de fração de tráfego legítimo afetada pela mitigação resultante. A implementação é disponibilizada como código aberto.

Keywords: Grafo de Conhecimento, Detecção de DDoS, Camada 7 HTTP, Sessão como Entidade Semântica, Raciocínio *Cross-Session*, IA Explicável

1. Introdução

1.1. Contexto e Motivação

Ataques de Negação de Serviço Distribuída (DDoS) na camada de aplicação mudaram qualitativamente na última década, e a escala recente é a evidência mais visível dessa mudança. O *survey* canônico de Tripathi e Hubballi reportou em 2021 que o volume de ataques de Camada 7 dobrava a cada trimestre, com um incidente sustentando 292.000 requisições por segundo durante treze dias [1]. Em agosto de 2023, o *HTTP/2 Rapid Reset* (CVE-2023-44487) inaugurou uma nova ordem de grandeza: Amazon Web Services, Cloudflare e Google divulgaram conjuntamente picos de 155, 201 e 398 milhões de requisições por segundo, respectivamente, mitigados naquele trimestre [2, 3, 4]. A operação que sustentou o pico de 398 milhões usou uma *botnet* de cerca de vinte mil máquinas — escala compatível com o que se observa rotineiramente em campanhas de Camada 7 sobre HTTP/HTTPS em 2024 e 2025, com mais de seis mil e quinhentos ataques *hyper-volumetric* bloqueados pela Cloudflare no segundo trimestre de 2025 (média de setenta e um ataques por dia) e crescimento de cento e vinte e nove por cento ano-a-ano especificamente em HTTP DDoS [5, 6].

A categoria central nesse novo regime é o **Slow HTTP DDoS distribuído**. Trata-se de uma família que inclui Slowloris e seus descendentes diretos, suas variantes de alta taxa (HULK, GoldenEye) e os ataques modernos sobre HTTP/2, como Rapid Reset, CONTINUATION Flood e MadeYouReset. O denominador comum dessa família é a *sub-limiaridade por sessão*: cada origem contribui pouco, o conjunto exaure o servidor. Vista isoladamente, cada requisição é indistinguível de uma legítima. A estrutura, os cabeçalhos e a frequência são compatíveis com tráfego normal. O sinal de ataque mora no padrão de uso da sessão e, mais ainda, no padrão estrutural entre sessões: a

duração das conexões mantidas pelo cliente, a entropia das rotas visitadas, a consistência do *fingerprint* do cliente, e a convergência de múltiplas origens no mesmo *endpoint* sob a mesma assinatura de cliente [7, 8, 9].

A literatura de detecção reconhece o problema, ao menos em parte. A meta-análise de Odusami et al. [10] sobre setenta e cinco estudos mostra que quarenta e sete por cento dos métodos usam *features* extraídas de sessões (taxa de requisições por usuário, duração média, contagem de operações por identidade). Em todos esses métodos, contudo, a sessão é reduzida a um vetor numérico antes de chegar ao classificador. O detector vê estatísticas, não vê a sessão. Não consegue, portanto, raciocinar sobre relações entre sessões, sobre identidades reaproveitadas, ou sobre padrões que só emergem quando duas sessões individualmente normais são analisadas em conjunto. Detectores recentes baseados em aprendizado de máquina para HTTP de Camada 7 reportam AUC (área sob a curva ROC) competitivo, mas seus autores admitem que a metodologia não foi validada em tempo real e que o resultado é binário, sem explicação ontológica [7].

Trabalhos contemporâneos buscam mitigar essa opacidade combinando representações em grafo com técnicas de IA explicável. Belcastro et al. [11] propõem o arcabouço KLAGÉ, que constrói grafos de conhecimento a partir de logs de rede, classifica nós via Graph-BERT, e gera relatórios em linguagem natural com auxílio de LIME e modelos de linguagem grandes. KLAGÉ inclui DDoS Slowloris entre os ataques avaliados e atinge $F1 = 84,1\%$ em *datasets* IoT de referência. Trata-se de evidência concreta de que o uso de grafos de conhecimento na detecção de tráfego anômalo se tornou prática viável. No entanto, o pipeline opera em nível de *nó de rede* (porta, IP, fluxo) em vez de modelar a sessão HTTP como entidade semântica raciocinável, captura coordenação entre nós implicitamente em *embeddings* aprendidos e não em relações simbólicas nomeadas, e termina no relatório textual sem articular uma política de mitigação cujo escopo seja derivado do próprio grafo. Esses três pontos delimitam a margem que o presente trabalho explora.

1.2. Definição do Problema

Tratamos a detecção de DDoS de Camada 7 sobre HTTP como um problema de *representação semântica de sessão*, e não de engenharia de features sobre estatísticas de sessão. A sessão HTTP é a unidade natural de comportamento de um cliente contra a aplicação. Também é o objeto que detectores existentes mais sumarizam e menos modelam semanticamente. Três deficiências do estado da arte motivam nossa abordagem:

1. **Sessão como *feature aggregate*, não como entidade.** A meta-análise de Odusami et al. [10] mostra que quarenta e sete por cento dos métodos de detecção de DDoS de Camada 7 usam estatísticas agregadas de sessão (taxa de requisições, duração, contagem de operações por identidade), mas nenhum dos setenta e cinco estudos catalogados trata a sessão como entidade semântica raciocinável, com identidade, alvo, comportamento e relações tipadas. Na prática, features de sessão viram números num vetor. O vetor não preserva o que torna a sessão útil em primeiro lugar: ela conecta múltiplas requisições, uma identidade, um endpoint alvo, e um histórico observável.
2. **Ausência de explicação ontológica.** Detectores baseados em aprendizado de máquina para ataques de Camada 7 HTTP reportam métricas competitivas, mas emitem rótulos binários sem fundamentação semântica. Os autores de um dos trabalhos mais recentes [7] notam explicitamente que sua metodologia não foi validada em tempo real e não produz explicação das decisões. Em operação real, um analista de segurança decide dezenas de vezes por hora entre bloquear um cliente, aplicar um desafio (*challenge*), ou degradar uma sessão. “Isto parece um ataque” não é decisão; é hesitação. A decisão exige saber qual endpoint está sob estresse, qual identidade está envolvida e qual sinal sustenta a hipótese.
3. **Ausência de raciocínio *cross-session* simbólico.** Raciocínio *cross-session* é a capacidade de relacionar múltiplas sessões através de identificadores compartilhados do cliente (identidade, *fingerprint* TLS, prefixo de IP), em vez de tratar cada sessão como caso independente. Campanhas coordenadas de Camada 7 dependem desse raciocínio. No caso da família Slow HTTP DDoS distribuído, o ataque envolve K origens distintas mantendo conexões parciais ou sustentadas contra o mesmo *endpoint*, com cada origem operando abaixo de qualquer limiar individual razoável. Cada sessão isolada tem assinatura fraca. O conjunto tem estrutura clara quando as sessões são analisadas em conjunto pelo cliente compartilhado, pela identidade reaproveitada ou pelo *fingerprint* TLS comum. O *fingerprint* TLS é uma assinatura computada a partir do *handshake* TLS, capaz de identificar o software cliente mesmo quando o IP de origem muda; JA3 e JA4 são as implementações conhecidas. Detectores que reduzem a sessão a *features* não conseguem raciocinar sobre esses laços entre sessões e perdem a campanha. Arcabouços recentes baseados em redes neurais sobre grafos, como

Graph-BERT em KLAGÉ [11], capturam parte dessa coordenação ao processar o grafo de tráfego, mas a relação entre sessões fica latente nos *embeddings*, sem expressão simbólica nomeada que possa ser auditada, consultada ou usada para derivar uma política de mitigação. Levantamentos recentes de grafos de conhecimento em cibersegurança [12] já reconheciam a lacuna: “*ainda é pouco explícito como implementar o grafo de conhecimento para enfrentar dificuldades industriais reais em situações de ciberataque e defesa*”. KGs construídos a partir de texto de inteligência de ameaças não capturam estrutura de tráfego em tempo de execução, e KGs alimentados em runtime por classificadores neurais entregam o veredicto mas não a justificativa formal que sustenta decisões de mitigação cirúrgica.

1.3. Questões de Pesquisa

Este trabalho aborda as seguintes questões de pesquisa:

- QP1:** Como uma ontologia OWL pode modelar a sessão HTTP, junto a *endpoint*, identidade e comportamento, com fidelidade suficiente para sustentar raciocínio sobre campanhas coordenadas de Camada 7 em vez de meramente agregar *features* ao nível de sessão?
- QP2:** Quais regras semânticas sobre tal grafo de conhecimento detectam ataques cuja assinatura individual em cada sessão fica abaixo dos limites de detectores baseados em *features* agregadas, mas cuja estrutura *cross-session* é discernível pela identidade ou pelo *fingerprint* compartilhado?
- QP3:** Como as decisões de detecção resultantes podem ser apresentadas como cadeias de evidências explícitas, ligando múltiplas sessões, identidades e *endpoints*, em formato auditável por analistas de segurança?
- QP4:** Qual é o ganho empírico do raciocínio semântico em nível de sessão sobre detecção baseada em *features* agregadas de sessão, controlando para o mesmo conjunto subjacente de atributos de tráfego?

1.4. Contribuições

Este trabalho não reivindica o uso pioneiro de grafos de conhecimento em detecção de ameaças; trabalhos como KLAGÉ [11] já demonstraram a viabilidade dessa abordagem. As contribuições deste artigo localizam-se em outro eixo, com quatro componentes principais:

1. **Sessão HTTP como entidade ontológica de primeira classe.** Uma ontologia OWL que modela a classe `ApplicationSession` com relações tipadas para identidade do cliente (`hasIdentity`), *endpoint* alvo (`targets`), comportamento exibido (`exhibitsBehavior`), correlação *cross-session* (`relatedTo`) e mitigação aplicável (`mitigatedBy`). A ontologia formaliza a família Slow HTTP DDoS distribuído como ataque coordenado de referência, com hierarquia que comporta variantes de conexão sustentada e variantes baseadas em taxa. Diferentemente de abordagens recentes baseadas em grafos [11], que operam em nível de nó de rede (porta, IP, fluxo), a unidade de raciocínio aqui é a sessão de aplicação, com identidade composta (`cookie`, `token`, `username`, `fingerprint` TLS) modelada explicitamente. A ontologia mantém alinhamento com STIX 2.1 e MITRE ATT&CK.
2. **Raciocínio simbólico nativo sobre o grafo, em tempo de execução.** Um *pipeline* que eleva requisições HTTP a instâncias ontológicas, populando sessões como nós persistentes do grafo, com a relação `relatedTo` instanciada a partir de identidade, `fingerprint` TLS e prefixo de cliente. As regras de detecção são fórmulas em SPARQL e SWRL, avaliadas por um *reasoner* OWL. O veredicto é a derivação simbólica, não a saída de um classificador interpretado a posteriori. Esse desenho contrasta tanto com grafos de cibersegurança construídos estaticamente a partir de texto de inteligência de ameaças [13, 14] quanto com abordagens neurais que classificam o grafo e explicam o resultado via técnicas pós-hoc [11].
3. **Cadeia de evidência simbólica acoplada à derivação automática de escopo de mitigação.** Quando uma regra dispara, o motor emite um veredicto acompanhado por uma cadeia de evidência exportável em JSON-LD e STIX 2.1, contendo as instâncias ontológicas envolvidas e os valores que satisfizeram a regra. A partir do conjunto mínimo de propriedades compartilhadas pelas sessões do *cluster*, o arcabouço deriva automaticamente o escopo da política de mitigação aplicada (por exemplo, o par (`fingerprint`, *padrão de endpoint*)), em contraste com defesas que aplicam controle global na rota e afetam tráfego legítimo. Esse acoplamento entre detecção, evidência e mitigação não é tratado por arcabouços acadêmicos contemporâneos baseados em KG [11], que terminam no relatório textual.
4. **Avaliação parametrizada pelo grau de distribuição da campanha, com métrica de dano colateral em tráfego legítimo.** A

avaliação isola, por ablação sobre o mesmo conjunto subjacente de atributos de tráfego, o ganho do raciocínio semântico em nível de sessão em relação a *baselines* de aprendizado de máquina que consomem as mesmas *features* agregadas. Além das métricas clássicas de classificação, reporta-se a fração de tráfego legítimo afetada pela mitigação resultante. Essa métrica é auto-reportada por produtos comerciais de *bot management* sem benchmark acadêmico público e não é usada sistematicamente na literatura de detecção de Camada 7. Uma implementação de referência em código aberto do *pipeline* completo é disponibilizada, endereçando a lacuna de implementação operacional quantificada por Odusami et al. [10].

1.5. Organização do Artigo

O restante deste artigo está organizado como segue. A Seção 2 revisa trabalhos relacionados em grafos de conhecimento para cibersegurança, em detecção de DDoS de Camada 7 sobre HTTP, e em modelagem de sessão para detecção de anomalias, posicionando nossa contribuição em relação às abordagens existentes mais próximas. A Seção 3 apresenta o arcabouço proposto, incluindo o *design* da ontologia centrada em sessão, o *pipeline* de construção do grafo de conhecimento em tempo de execução, e as regras de detecção semântica. A Seção 4 descreve a metodologia experimental, com a família Slow HTTP DDoS distribuído como caso instanciado, o conjunto de dados, os *baselines* baseados em *features* agregadas, e as métricas. A Seção 5 relata e discute os resultados, com ênfase na ablação que isola o ganho do raciocínio em nível de sessão e na análise da vantagem *cross-session*. A Seção 6 conclui e delineia direções para extensão.

2. Trabalhos Relacionados

2.1. Grafos de Conhecimento em Cibersegurança

A primeira onda de trabalhos em grafos de conhecimento aplicados à cibersegurança privilegiou a construção *estática* do grafo a partir de texto de inteligência de ameaças. Jia et al. [13] consolidaram um arcabouço prático em que entidades de CVEs, relatórios e *blogs* são extraídas, normalizadas e ligadas para produzir uma base de conhecimento consultável, útil para enriquecimento e correlação após o fato. Bonagiri et al. [14] reforçaram essa linha em 2024 com aplicações práticas em problemas reais de segurança, mantendo a mesma premissa: o grafo é um registro do que já se sabe sobre ameaças,

alimentado por texto e atualizado periodicamente. O levantamento de Liu et al. [12] sintetizou o estado da área e reconheceu explicitamente a lacuna entre a sofisticação do modelo e o uso operacional: “ainda é pouco explícito como implementar o grafo de conhecimento para enfrentar dificuldades industriais reais em situações de ciberataque e defesa”.

Trabalhos contemporâneos passaram a explorar grafos populados em tempo de execução a partir do próprio tráfego, com classificadores neurais sobre o grafo e técnicas de IA explicável acopladas. O exemplar mais próximo dessa linha é KLAGÉ, proposto por Belcastro et al. [11]. KLAGÉ constrói grafos de conhecimento a partir de logs e fluxos de rede, classifica nós com Graph-BERT, gera explicações com LIME e produz relatórios em linguagem natural por meio de modelos de linguagem grandes. Avaliado em *datasets* IoT de referência (RT-IoT2022 e CIC-IoT2023) sobre seis ataques — entre eles DDoS Slowloris — KLAGÉ atinge $F_1 = 84,1\%$, supera *baselines* GNN e BERT-based e disponibiliza código aberto. É a evidência mais clara de que grafos de conhecimento podem servir como mecanismo de detecção em runtime, e não apenas como registro pós-fato.

O presente trabalho parte da mesma premissa de runtime e diverge em três eixos. Primeiro, a unidade de raciocínio em KLAGÉ é o *nó de rede* (porta, IP, fluxo); aqui é a *sessão HTTP*, modelada como entidade ontológica de primeira classe, com identidade composta (cookie, *token*, *username*, *fingerprint* TLS). Segundo, KLAGÉ encoda o grafo via *embeddings* aprendidos (Graph-BERT) e explica a saída com LIME pós-hoc; este trabalho usa OWL com regras simbólicas em SPARQL/SWRL, e o veredicto é a derivação que satisfaz a regra. Terceiro, KLAGÉ termina no relatório textual; este trabalho deriva, a partir do discriminador do *cluster* detectado, o escopo da política de mitigação aplicada.

2.2. Detecção de Ataques DDoS de Camada 7 sobre HTTP

A detecção de DDoS na camada de aplicação HTTP organiza-se historicamente em três famílias. A primeira é o perfilamento estatístico de tráfego e sessão. Fernandes et al. [9] construíram perfis autônomos a partir de análise por componentes principais (PCA) sobre estatísticas agregadas de fluxo, com aplicação direta à detecção de anomalias em redes operacionais. Bharathi e Sukanesh [8] aplicaram a mesma ferramenta, PCA, a uma matriz comportamental por sessão, oferecendo uma das primeiras formulações que coloca a sessão no centro — ainda que como vetor de *features* agregadas, e não como entidade.

A segunda família agrega *features* de fluxo ou de sessão e as entrega a classificadores supervisionados. Kemp et al. [7] é o representante mais recente: emprega *Random Forest* e SVM sobre vetores de sessão e reporta AUC competitivo. Os autores notam explicitamente, no entanto, que a metodologia não foi validada em tempo real e que a saída é binária, sem explicação ontológica das decisões. Esta limitação é representativa da família: o classificador aprende a separar conjuntos no espaço de *features*, mas perde a estrutura entre sessões que justifica a separação.

A terceira família consolida o domínio em levantamentos de escala. Tripathi e Hubballi [1] sintetizaram em 2021 o estado da detecção e defesa em DDoS de camada de aplicação, com a observação — citada na Seção 1 — de que o volume de Camada 7 dobrava a cada trimestre. A taxonomia de Slow DoS Attacks de Cambiaso et al. [15] permanece como referência formal para a categoria que motiva nossa avaliação experimental.

O denominador comum das três famílias é o tratamento da sessão como agregado de *features*, e não como entidade raciocinável.

2.3. Modelagem de Sessão em Detecção de Anomalias

A meta-análise de Odusami et al. [10] sobre setenta e cinco estudos em detecção de DDoS de camada de aplicação fornece a evidência quantitativa mais clara sobre como a sessão é tratada. Em quarenta e sete por cento dos métodos catalogados, *features* extraídas de sessões compõem o vetor que alimenta o classificador: taxa de requisições por usuário, duração média, contagem de operações por identidade, entre outras. A redução é uniforme. Em nenhum dos estudos a sessão é mantida como objeto com identidade, alvo, comportamento e relações tipadas; ela é convertida em números antes do classificador. Como consequência, qualquer raciocínio sobre identidades reaproveitadas, sobre convergência de *fingerprints* em endpoints, ou sobre padrões que só emergem entre sessões fica fora do alcance do detector.

A literatura industrial de *bot management* reconhece e explora justamente esses padrões. Cloudflare documenta o uso de *fingerprints* JA4 para agrupar sessões aparentemente independentes por similaridade do *stack* TLS, em sua plataforma de Bot Management. DataDome reporta taxas de falso-positivo abaixo de 0,01% em CAPTCHAs servidos, sugerindo que a discriminação *cross-session* é viável em produção. Esses sistemas, entretanto, são proprietários, sem ontologia pública, sem benchmark acadêmico reproduzível e sem métricas de dano colateral em legítimos formalizadas em literatura revisada.

A formalização acadêmica auditável dessas práticas, e em particular a derivação automática de escopo de mitigação a partir de uma regra ontológica explícita, permanece em aberto.

2.4. Posicionamento deste Trabalho

Sintetizando, três fontes delimitam o espaço onde nossa contribuição se inscreve. Primeiro, KGs estáticos construídos a partir de texto [13, 14, 12] não capturam estrutura de tráfego em tempo de execução. Segundo, KGs populados em runtime e classificados com redes neurais [11] entregam veredicto e relatório, mas sem expressão simbólica das relações entre sessões e sem política de mitigação acoplada. Terceiro, detectores estatísticos e de aprendizado de máquina sobre *features* agregadas [9, 8, 7, 10] descartam a estrutura entre sessões antes mesmo do classificador.

Este trabalho ocupa a interseção que esses três grupos deixam vaga: a sessão HTTP modelada como entidade ontológica de primeira classe; o raciocínio sobre relações entre sessões expresso em formalismo simbólico nativo (OWL, SPARQL, SWRL) e auditável; e a derivação automática de escopo de mitigação a partir do discriminador do *cluster* detectado, com avaliação que reporta explicitamente o impacto sobre tráfego legítimo.

3. Abordagem Proposta

3.1. Visão Geral do Sistema

A Figura 1 apresenta a arquitetura geral do arcabouço proposto. O sistema é composto por quatro estágios: (i) ingestão de eventos HTTP do tráfego de aplicação, (ii) elevação dos eventos a instâncias ontológicas no grafo de conhecimento, com sessões como nós persistentes, (iii) execução de regras semânticas sobre o grafo para detecção de ataques nos três tipos modelados, e (iv) geração da cadeia de evidências que acompanha o veredicto.

3.2. Ontologia Centrada em Sessão

[*Esqueleto.*] Esta subseção define a ontologia OWL. A classe central é `ApplicationSession`, modelada como entidade de primeira classe com as seguintes relações tipadas:

`hasIdentity` liga uma sessão à identidade observada do cliente (cookie de sessão, *token* JWT, *username* em formulário de *login*, *fingerprint* TLS).

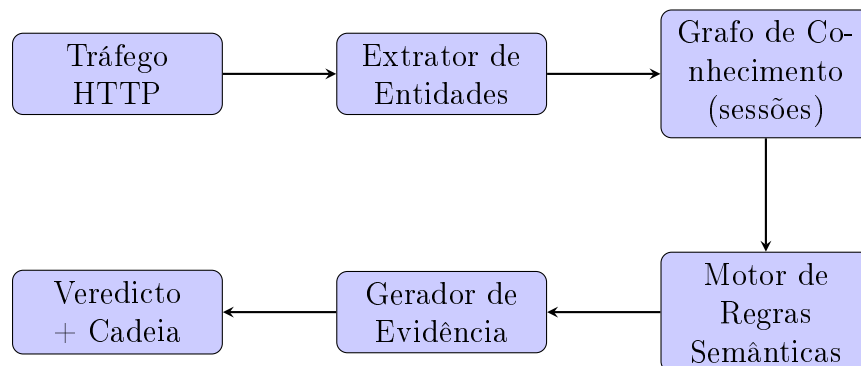


Figura 1: Arquitetura do arcabouço de detecção centrado em sessão.

`targets` liga uma sessão ao *endpoint* de aplicação alvo (`Endpoint`, com subclasses `AuthEndpoint`, `APIEndpoint`, `StaticAsset`).

`exhibitsBehavior` liga uma sessão a um perfil comportamental (`UserBehavior` ou `BotBehavior`).

`relatedTo` liga sessões entre si quando compartilham identidade, *fingerprint* ou prefixo de cliente; é a relação que habilita raciocínio *cross-session*.

`mitigatedBy` liga uma sessão ou ataque a uma política de mitigação aplicável (`RateLimit`, `Challenge`, `Block`).

Três especializações de *ataque coordenado* são definidas como subclasses de `ApplicationLayerAttack`: `CoordinatedHTTPFlood`, `CredentialStuffing` (subclasse específica de `LoginFlood`) e `CoordinatedAPIAbuse`. O denominador comum é a propriedade `exhibitsCrossSessionStructure`: múltiplas sessões compartilhando identidade, *fingerprint* TLS ou prefixo de IP enquanto convergem sobre o mesmo `Endpoint`. Cada especialização é caracterizada por um padrão semântico sobre as relações da sessão, não por um vetor de *features*.

3.3. Construção do Grafo em Tempo de Execução

[*Esqueleto.*] O *pipeline* de construção opera em três fases:

1. **Extração de entidades.** Para cada requisição HTTP observada, são criadas (ou atualizadas) instâncias de `HTTPRequest`, `ApplicationSession`, `Identity` e `Endpoint`.

2. **População de relações.** As relações `hasIdentity`, `targets`, `exhibitsBehavior` e `relatedTo` são instanciadas no grafo conforme as evidências do tráfico.
3. **Manutenção de janela.** Sessões e relações ficam ativas no grafo dentro de uma janela operacional configurável (por padrão, $W = 5$ minutos), após a qual são descarregadas para manter o grafo enxuto em ambiente de produção.

O Algoritmo 1 apresenta o pseudocódigo do *pipeline*.

Algorithm 1 Construção do Grafo de Conhecimento em Tempo de Execução

Require: Fluxo de requisições $\mathcal{R}$, ontologia $\mathcal{O}$, janela W

Ensure: Grafo de Conhecimento KG persistente em W

```

1:  $KG \leftarrow \emptyset$ 
2: for cada requisição  $r \in \mathcal{R}$  do
3:    $s \leftarrow \text{ResolverSessao}(r)$ 
4:    $i \leftarrow \text{ResolverIdentidade}(r)$ 
5:    $e \leftarrow \text{ResolverEndpoint}(r.\text{path})$ 
6:    $KG.\text{adicionar}(s, i, e)$ 
7:    $KG.\text{instanciarRelacao}(s, \text{hasIdentity}, i)$ 
8:    $KG.\text{instanciarRelacao}(s, \text{targets}, e)$ 
9:    $KG.\text{ligarSessoesPorIdentidade}(s)$ 
10:   $KG.\text{purgarFora}(W)$ 
11: end for
12: retornar  $KG$ 

```

3.4. Regras de Detecção Semântica

[*Esqueleto.*] Para cada uma das três especializações de ataque coordenado, definimos uma regra semântica sobre o grafo. Todas as regras dependem da relação `relatedTo` entre sessões; nenhuma é satisfeita por uma sessão isolada. As regras são expressas em SPARQL/SWRL no *deliverable* final do arcabouço; abaixo, as condições em forma narrativa:

- **Credential Stuffing** (Login Flood coordenado): existem múltiplas sessões $s_1, \dots, s_n$ ligadas por `relatedTo` (mesma identidade reaproveitada, mesmo *fingerprint* TLS ou mesmo prefixo de IP) dentro da janela W , todas com `targets` apontando para um `AuthEndpoint`, com razão agregada de falha de autenticação acima de um limiar τ_{fail} .

- **HTTP Flood distribuído:** existe um conjunto de sessões ligadas por `relatedTo` cuja taxa agregada de requisições contra um mesmo `Endpoint` excede τ_{rate} , com baixa entropia de rotas (`BotBehavior`) por sessão individual.
- **Abuso de API por múltiplas identidades:** existem múltiplas sessões com `hasIdentity` apontando para diferentes *tokens* mas `relatedTo` via *fingerprint* TLS ou prefixo de IP, todas atingindo o mesmo `APIEndpoint` a taxa que somada excede τ_{api} (mesmo que nenhuma sessão isolada exceda sua quota).

3.5. Cadeia de Evidência e Explicabilidade

[*Esqueleto.*] Quando uma regra dispara, o motor emite um veredicto acompanhado de uma cadeia de evidência que enumera explicitamente: a regra que disparou, as instâncias ontológicas envolvidas (sessões, identidades, *endpoint*), os valores observados das condições, e a política de mitigação sugerida. A cadeia é exportável em formato JSON-LD e STIX 2.1, compatível com integração em *Security Information and Event Management* (SIEM).

4. Metodologia Experimental

4.1. Cenários de Avaliação por Grau de Distribuição

[*Esqueleto.*] A hipótese central deste trabalho é que o raciocínio *cross-session* fornece vantagem mensurável especificamente sobre campanhas coordenadas, e que essa vantagem cresce com o **grau de distribuição** da campanha (número de sessões e de identidades simultâneas sob direção comum). Para isolar esse efeito, a avaliação é parametrizada por três cenários, em ordem crescente de distribuição:

Cenário A: Concentrado. Uma única origem (um `ClientIdentity`, um IP, um *fingerprint* TLS) gerando N requisições contra um endpoint. Representa o ataque clássico *single-source* de alto volume. **Hipótese:** detectores monoprotocolares por IP/sessão alcançam desempenho competitivo neste regime; o ganho do raciocínio *cross-session* é marginal.

Cenário B: Moderadamente Distribuído. K origens distintas ($10 \leq K \leq 100$) sob coordenação comum atacando o mesmo endpoint, cada uma operando dentro de seus limites individuais de *rate-limit*. Representa

botnets pequenas ou operações com proxies limitados. **Hipótese:** *baselines* de aprendizado de máquina sobre *features* agregadas começam a perder *recall*; raciocínio *cross-session* mantém detecção via relação *relatedTo* por *fingerprint* TLS ou prefixo de IP.

Cenário C: Altamente Distribuído. $K \geq 1000$ origens distintas em campanha coordenada (*credential stuffing* massivo, abuso de API por frota de *tokens*, HTTP Flood de baixa intensidade por botnet ampla). Cada sessão individual fica abaixo de qualquer limiar razoável. **Hipótese:** *baselines* caem para *recall* próximo do aleatório; raciocínio *cross-session* preserva detecção. **Este é o cenário onde a contribuição se sustenta.**

A *money figure* do paper é a curva de *recall* em função do grau de distribuição, com os *baselines* caindo de A para C enquanto o arcabouço proposto se mantém. A diferença empírica entre a curva de cada *baseline* e a do arcabouço completo no Cenário C é o resultado central a ser reportado.

4.2. Conjunto de Dados

[*Esqueleto.*] Para instrumentar os três cenários com controle preciso sobre o grau de distribuição, a avaliação primária utiliza **tráfego sintético** gerado pelo *pipeline* Python que acompanha este trabalho. A geração parametriza, para cada cenário: número de sessões legítimas concorrentes, número de origens da campanha, distribuição de identidades por origem, janela temporal e tipo de ataque (*credential stuffing*, HTTP Flood distribuído, abuso de API). Como avaliação secundária, subconjuntos de *datasets* públicos com campanhas instrumentadas (CICIDS2017 para Slowloris e Slow HTTP, CIC-DDoS2019 para componentes HTTP) serão usados para checar consistência. Dados reais anonimizados de operação ficam como direção futura sujeita a aprovação ética.

4.3. Configuração Experimental

[*Esqueleto.*] Descreverá ambiente experimental (*hardware*, *software*, parâmetros), a divisão treino/validação/teste, os procedimentos de calibração de limiares por cenário, e a janela W de manutenção do grafo. Cada cenário será executado $n \geq 30$ vezes com seeds diferentes para permitir estimação de intervalos de confiança.

4.4. Baselines

[Esqueleto.] A comparação experimental será feita contra três *baselines*, todos consumindo o mesmo conjunto subjacente de atributos de tráfego:

- **Perfilamento estatístico** (aproximação de Fernandes et al. [9]): PCA sobre estatísticas agregadas de sessão.
- **Matriz de comportamento** (aproximação de Bharathi e Sukanesh [8]): k -means sobre matriz de *features* por sessão.
- **Aprendizado de máquina supervisionado** (aproximação de Kemp et al. [7]): Random Forest e SVM sobre o mesmo vetor de *features* de sessão.

A escolha desses *baselines* é deliberada: todos consomem o mesmo material subjacente que nosso arcabouço, mas na forma de *features* agregadas por sessão. A ablação isola a contribuição da **representação semântica de sessão** e do raciocínio *cross-session*, não da disponibilidade dos atributos.

4.5. Métricas de Avaliação

[Esqueleto.] Três famílias de métricas são reportadas:

1. **Métricas padrão de classificação** por cenário: precisão, *recall*, F1, AUC, FPR.
2. **Recall por campanha** (não por sessão): para cada campanha coordenada instrumentada no *dataset*, qual fração foi detectada como pertencente à campanha (e não apenas sessões individuais isoladas). Esta métrica captura diretamente o ganho do raciocínio *cross-session*.
3. **Qualidade da explicação**: análise qualitativa das cadeias de evidência geradas, em termos de completude (cita todas as sessões correlacionadas) e acionabilidade (sugere mitigação coerente com o vetor identificado).

4.6. Análise de Ablação

[Esqueleto.] A análise central isola a contribuição de cada componente do arcabouço, em três configurações:

- (a) *Baseline* ML sem ontologia, alimentado pelas mesmas *features* agregadas de sessão.

- (b) Arcabouço com ontologia mas *sem* a relação `relatedTo` (sessões tratadas como nós isolados do grafo).
- (c) Arcabouço completo, com sessões ligadas por `relatedTo` via identidade, *fingerprint* TLS e prefixo de IP.

A diferença entre (a) e (c) é a contribuição total da abordagem; a diferença entre (b) e (c) é especificamente o ganho do raciocínio *cross-session*. Esperamos que essa última diferença seja pequena no Cenário A e dominante no Cenário C.

5. Resultados e Discussão

5.1. Desempenho Geral por Cenário

[*Esqueleto.*] Tabela comparativa de F1, FPR e AUC contra os três *baselines*, reportada separadamente para cada cenário de distribuição (A, B, C) e para cada um dos três ataques coordenados modelados, com intervalos de confiança.

5.2. Curva de Recall por Grau de Distribuição

[*Esqueleto.*] Esta é a **figura central** do paper: *recall* de cada método (três *baselines* e o arcabouço completo) plotado em função do grau de distribuição da campanha. A previsão é uma curva descendente íngreme para os *baselines* entre o Cenário B e o Cenário C, enquanto o arcabouço com raciocínio *cross-session* mantém *recall* estável.

5.3. Análise por Tipo de Ataque Coordenado

[*Esqueleto.*] Discussão por ataque: *credential stuffing*, HTTP Flood distribuído, e abuso de API por múltiplas identidades. Para cada um, identificar (i) o nível mínimo de distribuição a partir do qual os *baselines* começam a falhar e (ii) qual sinal estrutural específico (*fingerprint* TLS, prefixo de IP, identidade reaproveitada) sustenta a detecção pelo arcabouço.

5.4. Análise da Vantagem Cross-Session

[*Esqueleto.*] Subseção central do paper: medir, no Cenário C, a fração de campanhas coordenadas detectadas como tal (não apenas sessões individuais isoladas) pelo arcabouço proposto em comparação aos *baselines*. A hipótese a validar é que o raciocínio *cross-session* fornece vantagem mensurável precisamente quando o atacante distribui a operação por muitas sessões individualmente sub-limiare.

5.5. Avaliação da Qualidade da Explicação

[Esqueleto.] Análise qualitativa (amostragem de cadeias de evidência geradas) e, se viável, estudo com analistas de segurança avaliando completude e acionabilidade das explicações.

5.6. Significância Estatística

[Esqueleto.] Testes pareados (*paired t-test*, Wilcoxon) sobre as n execuções por cenário, com correção de Bonferroni para múltiplas comparações entre métodos.

5.7. Limitações

[Esqueleto.] Três limitações principais devem ser declaradas explicitamente:

1. **Vantagem regime-específica:** para ataques *single-source* de alto volume (Cenário A), métodos baseados em limiares estatísticos por IP ou por sessão alcançam desempenho comparável; o ganho marginal do raciocínio *cross-session* é pequeno nesse regime. O ganho cresce com o grau de distribuição da campanha. Este artigo não substitui defesas volumétricas; complementa-as.
2. **Dependência de identificação de sessão e identidade:** a abordagem assume capacidade de extrair sessões, identidades e *fingerprints* TLS dos eventos de tráfego. Em ambientes onde essa instrumentação é parcial ou ruidosa, o desempenho degrada.
3. **Custo de manutenção do grafo** em produção, sensibilidade ao parâmetro W (janela operacional), e generalização para protocolos além de HTTP (TLS *handshake*, DNS) ficam como direções de extensão.

6. Conclusão

[Esqueleto.] Síntese das contribuições: ontologia centrada em sessão, *pipeline* de KG em tempo de execução, regras semânticas explicáveis, ablação que isola a contribuição da camada semântica. Direções futuras incluem (i) extensão da ontologia para outros protocolos de aplicação (DNS, TLS *handshake*), (ii) integração com plataformas SOAR para resposta automatizada, (iii) avaliação em ambientes de produção com dados reais anonimizados, e (iv) estudos com analistas de segurança para validação operacional das cadeias de evidência.

Referências

- [1] N. Tripathi, N. Hubballi, Application layer denial-of-service attacks and defense mechanisms: A survey, *ACM Computing Surveys* 54 (4) (2021) 1–33. doi:10.1145/3448291.
- [2] Cloudflare, HTTP/2 zero-day vulnerability results in record-breaking DDoS attacks, Cloudflare Blog (Oct. 2023).
URL <https://blog.cloudflare.com/zero-day-rapid-reset-http2-record-breaking-ddos-attack>
- [3] Google Cloud, How it works: The novel HTTP/2 ‘Rapid Reset’ DDoS attack, Google Cloud Blog, google reports record-breaking peak of 398 million requests per second mitigated (Oct. 2023).
URL <https://cloud.google.com/blog/products/identity-security/how-it-works-the-novel-http2-rapid-reset-ddos-attack>
- [4] MITRE Corporation, CVE-2023-44487: HTTP/2 rapid reset attack, National Vulnerability Database, disclosed October 10, 2023; coordinated disclosure across multiple HTTP/2 implementations (2023).
URL <https://nvd.nist.gov/vuln/detail/CVE-2023-44487>
- [5] Cloudflare, Hyper-volumetric DDoS attacks skyrocket: Cloudflare’s 2025 Q2 DDoS threat report, Cloudflare Blog, reports 6,500+ hyper-volumetric attacks in Q2 2025 (avg. 71/day); HTTP DDoS grew 129% year-over-year (2025).
URL <https://blog.cloudflare.com/ddos-threat-report-for-2025-q2/>
- [6] Cloudflare, Targeted by 20.5 million DDoS attacks, up 358% year-over-year: Cloudflare’s 2025 Q1 DDoS threat report, Cloudflare Blog (2025).
URL <https://blog.cloudflare.com/ddos-threat-report-for-2025-q1/>
- [7] C. Kemp, C. Calvert, T. M. Khoshgoftaar, J. L. Leevy, An approach to application-layer DoS detection, *Journal of Big Data* 10 (1) (2023) 22. doi:10.1186/s40537-023-00699-3.
- [8] R. Bharathi, R. Sukanesh, A PCA based framework for detection of application layer DDoS attacks, *WSEAS Transactions on Information Science and Applications* 9 (12) (2012) 389–398.
- [9] G. Fernandes Jr., J. J. P. C. Rodrigues, M. L. Proença Jr., Autonomous profile-based anomaly detection system using principal component

- analysis and flow analysis, *Applied Soft Computing* 34 (2015) 513–525. doi:10.1016/j.asoc.2015.05.019.
- [10] M. Odusami, S. Misra, O. Abayomi-Alli, A. Abayomi-Alli, L. Fernández-Sanz, A survey and meta-analysis of application-layer distributed denial-of-service attack, *International Journal of Communication Systems* 33 (18) (2020) e4603. doi:10.1002/dac.4603.
 - [11] L. Belcastro, C. Carlucci, C. Cosentino, P. Liò, F. Marozzo, Enhancing network security using knowledge graphs and large language models for explainable threat detection, *Future Generation Computer Systems* 176 (2026) 108160, published online October 2025; received 29 April 2025; accepted 20 September 2025. doi:10.1016/j.future.2025.108160.
 - [12] K. Liu, F. Wang, Z. Ding, S. Liang, Z. Yu, Y. Zhou, Recent progress of using knowledge graph for cybersecurity, *Electronics* 11 (15) (2022) 2287. doi:10.3390/electronics11152287.
 - [13] Y. Jia, Y. Qi, H. Shang, R. Jiang, A. Li, A practical approach to constructing a knowledge graph for cybersecurity, *Engineering* 4 (1) (2018) 53–60. doi:10.1016/j.eng.2018.01.004.
 - [14] V. Bonagiri, P. Priyadharshini, P. PanneerSelvi, S. Sahitya, R. Swamy, P. Varalakshmi, Practical applications of cybersecurity: A research paper on knowledge graphs for real-world security solutions, in: 2024 4th Asian Conference on Innovation in Technology (ASIANCON), IEEE, 2024. doi:10.1109/ASIANCON62057.2024.10838144.
 - [15] E. Cambiaso, G. Papaleo, G. Chiola, M. Aiello, Slow DoS attacks: Definition and categorisation, in: *International Journal of Trust Management in Computing and Communications*, Vol. 1, Inderscience, 2013, pp. 300–319. doi:10.1504/IJTMCC.2013.056440.

Apêndice A. Detalhes da Ontologia

[Esqueleto.] A ontologia OWL completa será disponibilizada em formato Turtle e RDF/XML no repositório do projeto.

Apêndice B. Reprodutibilidade

[Esqueleto.] Código-fonte, *datasets* sintéticos e *scripts* experimentais serão disponibilizados em repositório público após a aceitação do artigo, sob licença permissiva.